MODULE *protocol*

Version 2 *Zcash P2P* protocol specification, following the draft *ZIP* "Version 2 *Zcash P2P* Network Protocol" (zcash/*zips* ≠ 1344).

Models connection setup and block synchronization over the stream layer of *streams.tla*: the *init* handshake on the dedicated handshake stream, protocol version negotiation, the long-lived block announcement streams (including the rule that at most one announcement stream of a type may be open per direction and the sender's option to replace a finished or reset stream), and headers-first synchronization over get-headers and get-blocks request streams, each carrying exactly one request and its response.

Blocks are heights on one linear chain: only the peer at the highest height extends it, and a peer that is behind downloads the heights it lacks.

Communication keeps the legacy model's message consumption discipline: a peer decides only from its own records; the only place both peers' records are written together is the transport itself (connection open/close, stream open, and the freeing of a stream slot once both sides have closed it).

Two boolean constants select how a receiver interprets the draft:

- *StrictSingleton*
  - TRUE a second open announcement stream of the same type from a peer is a *PROTOCOL_ERROR* (literal reading of "Announcement Streams")
  - FALSE the receiver accepts it and drains both
- *RefusePreHandshake*
  - TRUE announcement streams that arrive before the local handshake completes are refused with *REFUSED* ("Connection Handshake": MAY refuse)
  - FALSE they are buffered until it completes

Under *StrictSingleton* the invariant *NoHonestProtocolError* is violated: two conformant peers disconnect because stream independence lets a replacement stream's type byte arrive before the old stream's *FIN* or reset.

See documents/v2-*modeling.md* for the full write-up.

EXTENDS *TLC*, *Naturals*, *Sequences*, *FiniteSets*, *streams*, *records*

CONSTANT *InitialPeers*	set of peers
CONSTANT *MaxBlock*	maximum block height (initial and mined)
CONSTANT *MaxRestarts*	finish/reset of own announcement streams, per peer pair
CONSTANT *MaxHeaders*	headers per get-headers response (draft: 160)
CONSTANT *MaxBlocksPerRequest*	hashes per get-blocks request (draft: 128)
CONSTANT *MinVersion*	minimum protocol version of the draft (not yet assigned)
CONSTANT *Versions*	protocol versions a peer may advertise
CONSTANT *StrictSingleton*	see module comment
CONSTANT *RefusePreHandshake*	see module comment

VARIABLE *nodes*

$vars \triangleq \langle nodes \rangle$

See *README* for an explanation of symmetry reduction.

$Symmetry \triangleq Permutations(InitialPeers)$

For each initial peer construct a set of all other peers.

$$OtherPeers \triangleq [n \in InitialPeers \mapsto InitialPeers \setminus \{n\}]$$

$$Min(a,\, b) \triangleq \text{IF } a < b \text{ THEN } a \text{ ELSE } b$$
$$Max(a,\, b) \triangleq \text{IF } a > b \text{ THEN } a \text{ ELSE } b$$

$$\begin{array}{l}
Init \triangleq \\
\quad \exists\, blockset \in [InitialPeers \to (1 \,..\, MaxBlock)] : \\
\quad \exists\, verset \in [InitialPeers \to Versions] : \\
\qquad nodes = [i \in InitialPeers \mapsto [\\
\qquad\quad conn \mapsto [j \in OtherPeers[i] \mapsto \text{“none”}], \\
\qquad\quad close \mapsto [j \in OtherPeers[i] \mapsto NoCode], \\
\qquad\quad streams \mapsto [j \in OtherPeers[i] \mapsto [sid \in StreamIds(i,\, j) \mapsto NullStream]], \\
\qquad\quad init_sent \mapsto [j \in OtherPeers[i] \mapsto \text{FALSE}], \\
\qquad\quad init_recvd \mapsto [j \in OtherPeers[i] \mapsto \text{FALSE}], \\
\qquad\quad version \mapsto [j \in OtherPeers[i] \mapsto 0], \\
\qquad\quad peer_tip \mapsto [j \in OtherPeers[i] \mapsto 0], \\
\qquad\quad announced \mapsto [j \in OtherPeers[i] \mapsto 0], \\
\qquad\quad restarts \mapsto [j \in OtherPeers[i] \mapsto 0], \\
\qquad\quad want \mapsto [j \in OtherPeers[i] \mapsto \langle\rangle], \\
\qquad\quad score \mapsto [j \in OtherPeers[i] \mapsto 0], \\
\qquad\quad blocks \mapsto 1 \,..\, blockset[i], \\
\qquad\quad my_version \mapsto verset[i] \\
\qquad]]
\end{array}$$

— Helpers —

$$Height(n) \triangleq Cardinality(nodes[n].blocks)$$

$$Connected(n,\, m) \triangleq nodes[n].conn[m] \in \{\text{“initiator”},\ \text{“responder”}\}$$

The handshake is complete for n once it has both sent and received an *init* record (draft: “Handshake Sequence”, step 3).

$$HandshakeComplete(n,\, m) \triangleq nodes[n].init\_sent[m] \land nodes[n].init\_recvd[m]$$

$$S(n,\, m,\, sid) \triangleq nodes[n].streams[m][sid]$$

Slots n may use to open a new stream toward m (lowest free slot is taken).

$$FreeSlots(n,\, m) \triangleq \{k \in 1 \,..\, MaxStreams : S(n,\, m,\, \langle n,\, k\rangle).status = \text{“none”}\}$$
$$FreeSlot(n,\, m) \triangleq \text{CHOOSE } k \in FreeSlots(n,\, m) : \forall\, j \in FreeSlots(n,\, m) : k \leq j$$

Announcement streams of type t that n currently sends to m on.

$$\begin{array}{l}
LiveAnn(n,\, m,\, t) \triangleq \{sid \in StreamIds(n,\, m) : \\
\qquad\qquad \land\, sid[1] = n \\
\qquad\qquad \land\, S(n,\, m,\, sid).status = \text{“open”} \\
\qquad\qquad \land\, S(n,\, m,\, sid).rtype = t
\end{array}$$

$$\land\, S(n,\, m,\, sid).out = \text{“open”}\}$$

Request streams n has outstanding toward m.

$$\begin{aligned} OpenReq(n,\, m) \triangleq\ & \{sid \in StreamIds(n,\, m) : \\ & \land\, sid[1] = n \\ & \land\, S(n,\, m,\, sid).status = \text{“open”} \\ & \land\, S(n,\, m,\, sid).rtype \in RequestTypes\} \end{aligned}$$

The sequence of heights $a \,..\, b$ (empty when $b < a$).

$$Range(a,\, b) \triangleq [i \in 1 \,..\, (b - a + 1) \mapsto a + i - 1]$$

$$Contiguous(seq) \triangleq \forall\, i \in 1 \,..\, (Len(seq) - 1) : seq[i + 1] = seq[i] + 1$$

Heights of seq above h.

$$Above(seq,\, h) \triangleq SelectSeq(seq,\, \text{LAMBDA}\ x : x > h)$$

Handshake streams n knows about on its connection with m.

$$\begin{aligned} HandshakeStreams(n,\, m) \triangleq\ & \{sid \in StreamIds(n,\, m) : \\ & \land\, S(n,\, m,\, sid).status = \text{“open”} \\ & \land\, S(n,\, m,\, sid).rtype = HandshakeType\} \end{aligned}$$

Transport: once both peers have closed a stream its slot is freed.

$$\begin{aligned} & Settle(ns,\, n,\, m,\, sid) \triangleq \\ & \quad \text{IF}\ \land\, ns[n].streams[m][sid].status = \text{“closed”} \\ & \qquad \land\, ns[m].streams[n][sid].status = \text{“closed”} \\ & \qquad \text{THEN}\ [ns\ \text{EXCEPT}\ ![n].streams[m][sid] = NullStream, \\ & \qquad\qquad\qquad\qquad ![m].streams[n][sid] = NullStream] \\ & \qquad \text{ELSE}\ ns \end{aligned}$$

Transport: n closes the connection to m with an application error code.
Both peers observe the close; data in flight is lost.

$$\begin{aligned} & Close(ns,\, n,\, m,\, code) \triangleq \\ & \quad [ns\ \text{EXCEPT}\ ![n].conn[m] \quad\ = \text{“closed”}, \\ & \qquad\qquad\ ![n].close[m] \quad\ = code, \\ & \qquad\qquad\ ![n].streams[m] = [sid \in StreamIds(n,\, m) \mapsto NullStream], \\ & \qquad\qquad\ ![m].conn[n] \quad\ = \text{“closed”}, \\ & \qquad\qquad\ ![m].close[n] \quad\ = code, \\ & \qquad\qquad\ ![m].streams[n] = [sid \in StreamIds(n,\, m) \mapsto NullStream]] \end{aligned}$$

— Connection —

n dials m. The *QUIC* handshake is out of scope and establishment is atomic:
n becomes the initiator and m the responder of the application handshake.

$$\begin{aligned} & Connect \triangleq \\ & \quad \exists\, n \in InitialPeers : \\ & \qquad \exists\, m \in OtherPeers[n] : \end{aligned}$$

$\land\ nodes[n].conn[m] = \text{“none”}$
$\land\ nodes[m].conn[n] = \text{“none”}$
$\land\ nodes' = [nodes \text{ EXCEPT } ![n].conn[m] = \text{“initiator”},$
$\qquad\qquad\qquad\qquad\qquad\qquad ![m].conn[n] = \text{“responder”}]$

—— Handshake ——

The initiator opens the handshake stream and sends its *init* record on it (draft: “Handshake Sequence”, step 1). The type byte and the record are queued together: they travel in order on the same stream.

$OpenHandshakeStream \triangleq$
$\quad \exists\, n \in InitialPeers :$
$\quad\quad \exists\, m \in OtherPeers[n] :$
$\quad\quad\quad \land\ nodes[n].conn[m] = \text{“initiator”}$
$\quad\quad\quad \land\ \neg nodes[n].init\_sent[m]$
$\quad\quad\quad \land\ FreeSlots(n,\, m) \neq \{\}$
$\quad\quad\quad \land\ \text{LET } sid \triangleq \langle n,\, FreeSlot(n,\, m)\rangle$
$\quad\quad\quad\quad \text{IN} \quad nodes' = [nodes \text{ EXCEPT}$
$\quad\quad\quad\quad\quad\quad ![n].streams[m][sid] = OpenerStream(HandshakeType),$
$\quad\quad\quad\quad\quad\quad ![m].streams[n][sid] = PeerStream(HandshakeType,$
$\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad \langle MakeInit(nodes[n].my\_version,\, Height(n))\rangle),$
$\quad\quad\quad\quad\quad\quad ![n].init\_sent[m] \quad = \text{TRUE}]$

The responder sends its own *init* record on the handshake stream. It may do so as soon as it has seen the stream’s type byte, without waiting for the initiator’s *init* record (draft: “Handshake Sequence”).

$SendInitResponder \triangleq$
$\quad \exists\, n \in InitialPeers :$
$\quad\quad \exists\, m \in OtherPeers[n] :$
$\quad\quad\quad \exists\, sid \in HandshakeStreams(n,\, m) :$
$\quad\quad\quad\quad \land\ nodes[n].conn[m] = \text{“responder”}$
$\quad\quad\quad\quad \land\ \neg nodes[n].init\_sent[m]$
$\quad\quad\quad\quad \land\ nodes' = [nodes \text{ EXCEPT}$
$\quad\quad\quad\quad\quad\quad ![m].streams[n][sid] = PutData(@,\, MakeInit(nodes[n].my\_version,\, Height(n))),$
$\quad\quad\quad\quad\quad\quad ![n].init\_sent[m] \quad = \text{TRUE}]$

n consumes the type byte of a stream m opened. What happens depends on the type and on n’s own state:
0x00 from the responder, or a second 0x00 $\rightarrow$ *PROTOCOL_ERROR* (“Connection Handshake”)
any other stream before n’s handshake completed $\rightarrow$ refuse with *REFUSED*, or wait (*RefusePreHandshake*)
announcement duplicating an open one of type t $\rightarrow$ *PROTOCOL_ERROR*, or accept (*StrictSingleton*)
otherwise $\rightarrow$ record the type

$RecvTypeByte \triangleq$
$\quad \exists\, n \in InitialPeers :$

$$
\begin{array}{l}
\exists\, m \in \mathit{OtherPeers}[n] : \\
\quad \exists\, \mathit{sid} \in \mathit{StreamIds}(n,\, m) : \\
\qquad \text{LET } s \triangleq S(n,\, m,\, \mathit{sid}) \\
\qquad \text{IN} \\
\qquad \land\, \mathit{Connected}(n,\, m) \\
\qquad \land\, s.\mathit{status} = \text{“open”} \\
\qquad \land\, s.\mathit{rtype} = \text{“unknown”} \\
\qquad \land\, \mathit{Len}(s.\mathit{inq}) > 0 \\
\qquad \land\, \mathit{IsTypeByte}(\mathit{Head}(s.\mathit{inq})) \\
\qquad \land\, \text{LET } t \triangleq \mathit{Head}(s.\mathit{inq}).t \\
\qquad\qquad \mathit{accept} \triangleq [\mathit{nodes} \text{ EXCEPT } ![n].\mathit{streams}[m][\mathit{sid}].\mathit{rtype} = t, \\
\qquad\qquad\qquad\qquad\qquad\qquad\quad ![n].\mathit{streams}[m][\mathit{sid}].\mathit{inq} \;\; = \mathit{Tail}(@)] \\
\qquad\qquad \mathit{legalHs} \triangleq \mathit{nodes}[n].\mathit{conn}[m] = \text{“responder”} \land \mathit{HandshakeStreams}(n,\, m) = \{\} \\
\qquad\qquad \mathit{dup} \triangleq \exists\, o \in \mathit{StreamIds}(n,\, m) : \\
\qquad\qquad\qquad\qquad \land\, o \neq \mathit{sid} \\
\qquad\qquad\qquad\qquad \land\, o[1] = m \\
\qquad\qquad\qquad\qquad \land\, S(n,\, m,\, o).\mathit{status} = \text{“open”} \\
\qquad\qquad\qquad\qquad \land\, S(n,\, m,\, o).\mathit{rtype} = t \\
\qquad\quad \text{IN} \\
\qquad\quad \land\, t \in \mathit{StreamTypes} \\
\qquad\quad \land\, \lor\, \land\, t = \mathit{HandshakeType} \\
\qquad\qquad\qquad \land\, \lor\, \land\, \mathit{legalHs} \\
\qquad\qquad\qquad\qquad\quad \land\, \mathit{nodes}' = \mathit{accept} \\
\qquad\qquad\qquad\quad \lor\, \land\, \neg \mathit{legalHs} \\
\qquad\qquad\qquad\qquad\quad \land\, \mathit{nodes}' = \mathit{Close}(\mathit{nodes},\, n,\, m,\, \text{“PROTOCOL\_ERROR”}) \\
\qquad\qquad\quad \lor\, \land\, t \in \mathit{AnnTypes} \cup \mathit{RequestTypes} \\
\qquad\qquad\qquad \land\, \lor\, \land\, \neg \mathit{HandshakeComplete}(n,\, m) \\
\qquad\qquad\qquad\qquad\quad \land\, \mathit{RefusePreHandshake} \\
\qquad\qquad\qquad\qquad\quad \land\, \mathit{nodes}' = \mathit{Settle}([\mathit{nodes} \text{ EXCEPT} \\
\qquad\qquad\qquad\qquad\qquad\qquad\quad ![n].\mathit{streams}[m][\mathit{sid}] = \mathit{ClosedStream}, \\
\qquad\qquad\qquad\qquad\qquad\qquad\quad ![m].\mathit{streams}[n][\mathit{sid}] = \mathit{Refuse}(@,\, t,\, \text{“REFUSED”})], \\
\qquad\qquad\qquad\qquad\qquad\quad n,\, m,\, \mathit{sid}) \\
\qquad\qquad\qquad\quad \lor\, \land\, \mathit{HandshakeComplete}(n,\, m) \\
\qquad\qquad\qquad\qquad\quad \land\, \lor\, \land\, t \in \mathit{AnnTypes} \land \mathit{StrictSingleton} \land \mathit{dup} \\
\qquad\qquad\qquad\qquad\qquad\quad \land\, \mathit{nodes}' = \mathit{Close}(\mathit{nodes},\, n,\, m,\, \text{“PROTOCOL\_ERROR”}) \\
\qquad\qquad\qquad\qquad\qquad \lor\, \land\, \neg(t \in \mathit{AnnTypes} \land \mathit{StrictSingleton} \land \mathit{dup}) \\
\qquad\qquad\qquad\qquad\qquad\quad \land\, \mathit{nodes}' = \mathit{accept}
\end{array}
$$

n consumes m's *init* record from the handshake stream (draft: “Handshake Validation”):

- second *init* record → *PROTOCOL_ERROR*
- version below *MinVersion* → *OBSOLETE*
- otherwise → handshake received; negotiated *version* = *min*

$$
\begin{array}{l}
\mathit{RecvInit} \triangleq \\
\quad \exists\, n \in \mathit{InitialPeers} :
\end{array}
$$

```
    ∃ m ∈ OtherPeers[n] :
      ∃ sid ∈ HandshakeStreams(n, m) :
        LET s ≜ S(n, m, sid)
        IN
        ∧ Connected(n, m)
        ∧ Len(s.inq) > 0
        ∧ Head(s.inq).kind = "init"
        ∧ LET msg ≜ Head(s.inq)
          IN
          ∨ ∧ nodes[n].init_recvd[m]
            ∧ nodes' = Close(nodes, n, m, "PROTOCOL_ERROR")
          ∨ ∧ ¬nodes[n].init_recvd[m]
            ∧ msg.version < MinVersion
            ∧ nodes' = Close(nodes, n, m, "OBSOLETE")
          ∨ ∧ ¬nodes[n].init_recvd[m]
            ∧ msg.version ≥ MinVersion
            ∧ nodes' = [nodes EXCEPT
                  ![n].streams[m][sid].inq = Tail(@),
                  ![n].init_recvd[m]       = TRUE,
                  ![n].version[m]          = Min(nodes[n].my_version, msg.version),
                  ![n].peer_tip[m]         = msg.start_height]
```

— Announcement streams —

After its handshake completes, n opens one announcement stream of each type it has none open for (draft: "Announcement Streams"). This is also how a replacement is opened after a finish, reset, or refusal. A reset can overtake announcements still in flight, so the sender forgets what it announced and re-announces its tip on the new stream.

```
OpenAnnouncementStream ≜
    ∃ n ∈ InitialPeers :
      ∃ m ∈ OtherPeers[n] :
        ∃ t ∈ AnnTypes :
          ∧ Connected(n, m)
          ∧ HandshakeComplete(n, m)
          ∧ LiveAnn(n, m, t) = {}
          ∧ FreeSlots(n, m) ≠ {}
          ∧ LET sid ≜ ⟨n, FreeSlot(n, m)⟩
            IN  nodes' = [nodes EXCEPT
                  ![n].streams[m][sid] = OpenerStream(t),
                  ![m].streams[n][sid] = PeerStream(t, ⟨⟩),
                  ![n].announced[m]    = 0]
```

n announces its current tip to m when it has grown since the last

announcement (draft: “Block Announcements”, header announcement).

$$
\begin{array}{l}
\mathit{SendAnnouncement} \triangleq \\
\quad \exists\, n \in \mathit{InitialPeers} : \\
\quad\quad \exists\, m \in \mathit{OtherPeers}[n] : \\
\quad\quad\quad \exists\, \mathit{sid} \in \mathit{LiveAnn}(n,\, m,\, \mathit{BlockAnnType}) : \\
\quad\quad\quad\quad \land\, \mathit{nodes}[n].\mathit{announced}[m] < \mathit{Height}(n) \\
\quad\quad\quad\quad \land\, \mathit{nodes}' = [\mathit{nodes}\ \text{EXCEPT} \\
\quad\quad\quad\quad\quad\quad ![m].\mathit{streams}[n][\mathit{sid}] = \mathit{PutData}(@,\, \mathit{MakeHeaderAnnouncement}(\mathit{Height}(n))), \\
\quad\quad\quad\quad\quad\quad ![n].\mathit{announced}[m] \;\; = \mathit{Height}(n)]
\end{array}
$$

n consumes a header announcement from m. Announcements are only processed once n’s handshake is complete (draft: “Connection Handshake”).

$$
\begin{array}{l}
\mathit{RecvAnnouncement} \triangleq \\
\quad \exists\, n \in \mathit{InitialPeers} : \\
\quad\quad \exists\, m \in \mathit{OtherPeers}[n] : \\
\quad\quad\quad \exists\, \mathit{sid} \in \mathit{StreamIds}(n,\, m) : \\
\quad\quad\quad\quad \text{LET } s \triangleq S(n,\, m,\, \mathit{sid}) \\
\quad\quad\quad\quad \text{IN} \\
\quad\quad\quad\quad \land\, \mathit{Connected}(n,\, m) \\
\quad\quad\quad\quad \land\, \mathit{HandshakeComplete}(n,\, m) \\
\quad\quad\quad\quad \land\, s.\mathit{status} = \text{“open”} \\
\quad\quad\quad\quad \land\, s.\mathit{rtype} \in \mathit{AnnTypes} \\
\quad\quad\quad\quad \land\, \mathit{Len}(s.\mathit{inq}) > 0 \\
\quad\quad\quad\quad \land\, \mathit{Head}(s.\mathit{inq}).\mathit{kind} = \text{“header”} \\
\quad\quad\quad\quad \land\, \mathit{nodes}' = [\mathit{nodes}\ \text{EXCEPT} \\
\quad\quad\quad\quad\quad\quad ![n].\mathit{streams}[m][\mathit{sid}].\mathit{inq} = \mathit{Tail}(@), \\
\quad\quad\quad\quad\quad\quad ![n].\mathit{peer\_tip}[m] \;\;\;\;\;\;\;\; = \mathit{Max}(@,\, \mathit{Head}(s.\mathit{inq}).\mathit{height})]
\end{array}
$$

n finishes one of its announcement streams. The draft allows a sender to open a replacement afterwards; why it finished (backpressure, internal restart) is not modeled. Bounded by *MaxRestarts* to keep liveness meaningful.

$$
\begin{array}{l}
\mathit{FinishAnnouncementStream} \triangleq \\
\quad \exists\, n \in \mathit{InitialPeers} : \\
\quad\quad \exists\, m \in \mathit{OtherPeers}[n] : \\
\quad\quad\quad \exists\, \mathit{sid} \in \mathit{LiveAnn}(n,\, m,\, \mathit{BlockAnnType}) : \\
\quad\quad\quad\quad \land\, \mathit{nodes}[n].\mathit{restarts}[m] < \mathit{MaxRestarts} \\
\quad\quad\quad\quad \land\, \mathit{nodes}' = \mathit{Settle}([\mathit{nodes}\ \text{EXCEPT} \\
\quad\quad\quad\quad\quad\quad ![n].\mathit{streams}[m][\mathit{sid}] = \mathit{ClosedStream}, \\
\quad\quad\quad\quad\quad\quad ![m].\mathit{streams}[n][\mathit{sid}] = \mathit{PutData}(@,\, \mathit{FIN}), \\
\quad\quad\quad\quad\quad\quad ![n].\mathit{restarts}[m] \;\;\;\;\;\; = @ + 1], \\
\quad\quad\quad\quad\quad\quad n,\, m,\, \mathit{sid})
\end{array}
$$

Same as *FinishAnnouncementStream* but via *RESET_STREAM*, which can overtake records still in flight on that stream.

$$
\begin{array}{l}
\mathit{ResetAnnouncementStream} \triangleq \\
\quad \exists\, n \in \mathit{InitialPeers} :
\end{array}
$$

```
        ∃ m ∈ OtherPeers[n] :
          ∃ sid ∈ LiveAnn(n, m, BlockAnnType) :
            ∧ nodes[n].restarts[m] < MaxRestarts
            ∧ nodes' = Settle([nodes EXCEPT
                    ![n].streams[m][sid] = ClosedStream,
                    ![m].streams[n][sid] = PutReset(@, "INTERNAL_ERROR"),
                    ![n].restarts[m]     = @ + 1],
                  n, m, sid)
```

— Request streams: headers-first synchronization —

n is behind m (from m's *init* or announcements), has nothing queued to download and no request outstanding: it asks m for headers after its own tip (draft: "Headers-First Synchronization", step 1). The request and the *FIN* of n's sending direction are written together.

```
SendGetHeaders ≜
    ∃ n ∈ InitialPeers :
      ∃ m ∈ OtherPeers[n] :
        ∧ Connected(n, m)
        ∧ HandshakeComplete(n, m)
        ∧ nodes[n].peer_tip[m] > Height(n)
        ∧ nodes[n].want[m] = ⟨⟩
        ∧ OpenReq(n, m) = {}
        ∧ FreeSlots(n, m) ≠ {}
        ∧ LET sid ≜ ⟨n, FreeSlot(n, m)⟩
          IN  nodes' = [nodes EXCEPT
                ![n].streams[m][sid] = RequesterStream(GetHeadersType),
                ![m].streams[n][sid] = PeerStream(GetHeadersType,
                                         ⟨MakeGetHeaders(Height(n)), FIN⟩)]
```

n serves a get-headers request from m: the headers after the locator, at most *MaxHeaders* of them, then *FIN*. Serving may start as soon as the request is complete, before m's *FIN* has been consumed (draft: "Request Streams").

```
ServeGetHeaders ≜
    ∃ n ∈ InitialPeers :
      ∃ m ∈ OtherPeers[n] :
        ∃ sid ∈ StreamIds(n, m) :
          LET s ≜ S(n, m, sid)
          IN
           ∧ Connected(n, m)
           ∧ HandshakeComplete(n, m)
           ∧ s.status = "open"
           ∧ s.rtype = GetHeadersType
```

```
                ∧ s.out = "open"
                ∧ Len(s.inq) > 0
                ∧ Head(s.inq).kind = "get-headers"
                ∧ LET loc  ≜ Head(s.inq).locator
                      hdrs ≜ Range(loc + 1, Min(loc + MaxHeaders, Height(n)))
                  IN  nodes' = [nodes EXCEPT
                        ![n].streams[m][sid] = Collapse([@ EXCEPT !.inq = Tail(@),
                                                                  !.out = "finished"]),
                        ![m].streams[n][sid] = PutData(PutData(@, MakeHeaders(hdrs)), FIN)]
```

n consumes a headers response from m (draft: "get-headers"):
more than *MaxHeaders*, or not contiguous → discard, misbehavior penalty
otherwise → queue the heights n lacks

```
RecvHeaders ≜
  ∃ n ∈ InitialPeers :
    ∃ m ∈ OtherPeers[n] :
      ∃ sid ∈ OpenReq(n, m) :
        LET s ≜ S(n, m, sid)
        IN
         ∧ Connected(n, m)
         ∧ s.rtype = GetHeadersType
         ∧ Len(s.inq) > 0
         ∧ Head(s.inq).kind = "headers"
         ∧ LET hdrs ≜ Head(s.inq).heights
           IN
            ∨ ∧ Len(hdrs) ≤ MaxHeaders ∧ Contiguous(hdrs)
              ∧ nodes' = [nodes EXCEPT
                    ![n].streams[m][sid].inq = Tail(@),
                    ![n].want[m]             = Above(hdrs, Height(n))]
            ∨ ∧ ¬(Len(hdrs) ≤ MaxHeaders ∧ Contiguous(hdrs))
              ∧ nodes' = [nodes EXCEPT
                    ![n].streams[m][sid].inq = Tail(@),
                    ![n].score[m]            = @ + 20]
```

n requests the next batch of blocks it wants from m, at most *MaxBlocksPerRequest* of them (draft: "get-blocks").

```
SendGetBlocks ≜
  ∃ n ∈ InitialPeers :
    ∃ m ∈ OtherPeers[n] :
      ∧ Connected(n, m)
      ∧ nodes[n].want[m] ≠ ⟨⟩
      ∧ OpenReq(n, m) = {}
      ∧ FreeSlots(n, m) ≠ {}
      ∧ LET sid   ≜ ⟨n, FreeSlot(n, m)⟩
            batch ≜ SubSeq(nodes[n].want[m], 1,
```

$$
\begin{array}{l}
\qquad\qquad\qquad\qquad Min(MaxBlocksPerRequest,\ Len(nodes[n].want[m])))\\
\quad \text{IN}\quad nodes' = [nodes\ \text{EXCEPT}\\
\qquad\qquad ![n].streams[m][sid] = RequesterStream(GetBlocksType),\\
\qquad\qquad ![m].streams[n][sid] = PeerStream(GetBlocksType,\\
\qquad\qquad\qquad\qquad\qquad \langle MakeGetBlocks(batch),\ FIN\rangle)]
\end{array}
$$

n serves a get-blocks request from m. It may finish after any complete entry, delivering fewer blocks than requested (draft: "get-blocks"); the requester re-requests the remainder.

$$
\begin{array}{l}
ServeGetBlocks \triangleq\\
\quad \exists\, n \in InitialPeers :\\
\quad\quad \exists\, m \in OtherPeers[n] :\\
\quad\quad\quad \exists\, sid \in StreamIds(n, m) :\\
\quad\quad\quad\quad \text{LET } s \triangleq S(n, m, sid)\\
\quad\quad\quad\quad \text{IN}\\
\quad\quad\quad\quad \land\ Connected(n, m)\\
\quad\quad\quad\quad \land\ HandshakeComplete(n, m)\\
\quad\quad\quad\quad \land\ s.status = \text{“open”}\\
\quad\quad\quad\quad \land\ s.rtype = GetBlocksType\\
\quad\quad\quad\quad \land\ s.out = \text{“open”}\\
\quad\quad\quad\quad \land\ Len(s.inq) > 0\\
\quad\quad\quad\quad \land\ Head(s.inq).kind = \text{“get-blocks”}\\
\quad\quad\quad\quad \land\ \text{LET } asked \triangleq Head(s.inq).heights\\
\quad\quad\quad\quad\quad\quad\quad held \triangleq SelectSeq(asked,\ \text{LAMBDA } h : h \in nodes[n].blocks)\\
\quad\quad\quad\quad\quad \text{IN}\\
\quad\quad\quad\quad\quad \exists\, k \in 1 \,..\, Len(held) :\\
\quad\quad\quad\quad\quad\quad nodes' = [nodes\ \text{EXCEPT}\\
\quad\quad\quad\quad\quad\quad\quad ![n].streams[m][sid] = Collapse([@\ \text{EXCEPT}\ !.inq = Tail(@),\\
\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad !.out = \text{“finished”}]),\\
\quad\quad\quad\quad\quad\quad\quad ![m].streams[n][sid] = PutData(PutData(@,\ MakeBlocks(SubSeq(held, 1, k))),\ FIN)]
\end{array}
$$

n consumes delivered blocks from m and extends its chain with them.

$$
\begin{array}{l}
RecvBlocks \triangleq\\
\quad \exists\, n \in InitialPeers :\\
\quad\quad \exists\, m \in OtherPeers[n] :\\
\quad\quad\quad \exists\, sid \in OpenReq(n, m) :\\
\quad\quad\quad\quad \text{LET } s \triangleq S(n, m, sid)\\
\quad\quad\quad\quad \text{IN}\\
\quad\quad\quad\quad \land\ Connected(n, m)\\
\quad\quad\quad\quad \land\ s.rtype = GetBlocksType\\
\quad\quad\quad\quad \land\ Len(s.inq) > 0\\
\quad\quad\quad\quad \land\ Head(s.inq).kind = \text{“blocks”}\\
\quad\quad\quad\quad \land\ \text{LET } got \triangleq Head(s.inq).heights\\
\quad\quad\quad\quad\quad\quad\quad blocks \triangleq nodes[n].blocks \cup \{got[i] : i \in 1 \,..\, Len(got)\}\\
\quad\quad\quad\quad\quad \text{IN}\quad nodes' = [nodes\ \text{EXCEPT}
\end{array}
$$

$$
\begin{aligned}
&!{[}n{]}.streams[m][sid].inq = Tail(@),\\
&!{[}n{]}.blocks \quad\quad\quad\quad\quad\quad = blocks,\\
&!{[}n{]}.want[m] \quad\quad\quad\quad\quad\quad = Above(nodes[n].want[m],\ Cardinality(blocks))]
\end{aligned}
$$

— Stream teardown —

n consumes a *FIN*: the peer's sending direction is over, and the stream closes once n's own direction is too. On a request stream the *FIN* may arrive after the response was already served; it is never "data after a complete request". A *FIN* before the type byte, or on the handshake stream, is handled per the draft (*PROTOCOL_ERROR*; graceful close) although honest peers never produce either.

$$
\begin{aligned}
&RecvFin \triangleq\\
&\quad \exists\, n \in InitialPeers :\\
&\quad\quad \exists\, m \in OtherPeers[n] :\\
&\quad\quad\quad \exists\, sid \in StreamIds(n,\ m) :\\
&\quad\quad\quad\quad \text{LET } s \triangleq S(n,\ m,\ sid)\\
&\quad\quad\quad\quad \text{IN}\\
&\quad\quad\quad\quad \land\ Connected(n,\ m)\\
&\quad\quad\quad\quad \land\ s.status = \text{“open”}\\
&\quad\quad\quad\quad \land\ Len(s.inq) > 0\\
&\quad\quad\quad\quad \land\ IsFin(Head(s.inq))\\
&\quad\quad\quad\quad \land\ \lor\ \land\ s.rtype = \text{“unknown”}\\
&\quad\quad\quad\quad\quad\quad\quad \land\ nodes' = Close(nodes,\ n,\ m,\ \text{“PROTOCOL\_ERROR”})\\
&\quad\quad\quad\quad\quad\quad \lor\ \land\ s.rtype = HandshakeType\\
&\quad\quad\quad\quad\quad\quad\quad \land\ nodes' = Close(nodes,\ n,\ m,\ \text{“NO\_ERROR”})\\
&\quad\quad\quad\quad\quad\quad \lor\ \land\ s.rtype \in AnnTypes \cup RequestTypes\\
&\quad\quad\quad\quad\quad\quad\quad \land\ nodes' = Settle([nodes \text{ EXCEPT}\\
&\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad !{[}n{]}.streams[m][sid] = Collapse([@ \text{ EXCEPT } !.inq = Tail(@),\\
&\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad !.in\_done = \text{TRUE}])],\\
&\quad\quad\quad\quad\quad\quad\quad\quad\quad\quad n,\ m,\ sid)
\end{aligned}
$$

n observes a *RESET_STREAM* from m. Queued data on that stream is discarded.

$$
\begin{aligned}
&RecvReset \triangleq\\
&\quad \exists\, n \in InitialPeers :\\
&\quad\quad \exists\, m \in OtherPeers[n] :\\
&\quad\quad\quad \exists\, sid \in StreamIds(n,\ m) :\\
&\quad\quad\quad\quad \text{LET } s \triangleq S(n,\ m,\ sid)\\
&\quad\quad\quad\quad \text{IN}\\
&\quad\quad\quad\quad \land\ Connected(n,\ m)\\
&\quad\quad\quad\quad \land\ s.status = \text{“open”}\\
&\quad\quad\quad\quad \land\ s.in\_reset \neq NoCode\\
&\quad\quad\quad\quad \land\ \lor\ \land\ s.rtype = HandshakeType\\
&\quad\quad\quad\quad\quad\quad\quad \land\ nodes' = Close(nodes,\ n,\ m,\ \text{“NO\_ERROR”})
\end{aligned}
$$

```
        ∨ ∧ s.rtype ≠ HandshakeType
          ∧ nodes' = Settle([nodes EXCEPT ![n].streams[m][sid] = ClosedStream],
                            n, m, sid)
```

n observes a *STOP_SENDING* from m on a stream n is sending on, and resets its sending direction as the draft recommends ("Request Streams").

```
RecvStop ≜
  ∃ n ∈ InitialPeers :
    ∃ m ∈ OtherPeers[n] :
      ∃ sid ∈ StreamIds(n, m) :
        LET s ≜ S(n, m, sid)
        IN
         ∧ Connected(n, m)
         ∧ sid[1] = n
         ∧ s.status = "open"
         ∧ s.out = "open"
         ∧ s.stop ≠ NoCode
         ∧ nodes' = Settle([nodes EXCEPT
                 ![n].streams[m][sid] = ClosedStream,
                 ![m].streams[n][sid] = PutReset(@, s.stop)],
             n, m, sid)
```

— Chain growth —

The peer at the chain tip finds a new block, giving it something to announce. Only the highest peer extends the chain, which keeps the model a single linear chain that the others catch up with.

```
MineBlock ≜
  ∃ n ∈ InitialPeers :
    ∧ Height(n) < MaxBlock
    ∧ ∀ m ∈ OtherPeers[n] : Height(m) ≤ Height(n)
    ∧ nodes' = [nodes EXCEPT ![n].blocks = @ ∪ {Height(n) + 1}]
```

```
Next ≜
  ∨ Connect
  ∨ OpenHandshakeStream
  ∨ SendInitResponder
  ∨ RecvTypeByte
  ∨ RecvInit
  ∨ OpenAnnouncementStream
  ∨ SendAnnouncement
  ∨ RecvAnnouncement
  ∨ FinishAnnouncementStream
```

$\lor$ *ResetAnnouncementStream*
$\lor$ *SendGetHeaders*
$\lor$ *ServeGetHeaders*
$\lor$ *RecvHeaders*
$\lor$ *SendGetBlocks*
$\lor$ *ServeGetBlocks*
$\lor$ *RecvBlocks*
$\lor$ *RecvFin*
$\lor$ *RecvReset*
$\lor$ *RecvStop*
$\lor$ *MineBlock*

Weak fairness on *Next*: as long as any action is enabled, one is taken. Mining and restarts are bounded, so every behaviour quiesces and every enabled receive eventually happens. The one loop that is not bounded is a responder refusing pre-handshake announcement streams while the sender keeps reopening them; fairness on *RecvInit* guarantees the responder eventually processes the *init* record that ends that loop.

$Spec \triangleq$
$Init$
$\land \Box[Next]_{vars}$
$\land \mathrm{WF}_{vars}(Next)$
$\land \mathrm{WF}_{vars}(RecvInit)$

— Liveness —

Every connection eventually completes its handshake (or has been closed).

$HandshakeCompletes \triangleq$
$\Diamond\Box\forall\, n \in InitialPeers : \forall\, m \in OtherPeers[n] :$
$nodes[n].conn[m] = \text{“closed”} \lor HandshakeComplete(n,\ m)$

Every peer eventually learns every connected peer's final tip.

$AnnouncementsFlow \triangleq$
$\Diamond\Box\forall\, n \in InitialPeers : \forall\, m \in OtherPeers[n] :$
$nodes[n].conn[m] = \text{“closed”} \lor nodes[n].peer\_tip[m] = Height(m)$

Eventually all peers hold the same chain.

$EventualConsensus \triangleq$
$\Diamond\Box\forall\, i,\ j \in InitialPeers : nodes[i].blocks = nodes[j].blocks$

— Safety invariants —

$TypeOK \triangleq$
$\forall\, n \in InitialPeers : \forall\, m \in OtherPeers[n] :$

$\land\ nodes[n].conn[m] \in \{\text{“none”},\ \text{“initiator”},\ \text{“responder”},\ \text{“closed”}\}$
$\land\ nodes[n].close[m] \in ErrorCodes \cup \{NoCode\}$
$\land\ nodes[n].version[m] \in Versions \cup \{0\}$
$\land\ nodes[n].restarts[m] \in 0\ ..\ MaxRestarts$
$\land\ nodes[n].score[m] \in Nat$
$\land\ \forall\, sid \in StreamIds(n,\ m) :$
$\quad \land\ S(n,\ m,\ sid).status \in \{\text{“none”},\ \text{“open”},\ \text{“closed”}\}$
$\quad \land\ S(n,\ m,\ sid).rtype \in StreamTypes \cup \{\text{“unknown”}\}$
$\quad \land\ S(n,\ m,\ sid).out \in \{\text{“open”},\ \text{“finished”},\ \text{“reset”},\ \text{“na”}\}$

A stream slot is free on one side exactly when it is free on the other.

$SlotsConsistent \triangleq$
$\forall\, n \in InitialPeers : \forall\, m \in OtherPeers[n] : \forall\, sid \in StreamIds(n,\ m) :$
$(S(n,\ m,\ sid).status = \text{“none”}) = (S(m,\ n,\ sid).status = \text{“none”})$

At most one handshake stream per connection, opened by the initiator (draft: “Connection Handshake”).

$OneHandshakeStream \triangleq$
$\forall\, n \in InitialPeers : \forall\, m \in OtherPeers[n] :$
$\land\ Cardinality(HandshakeStreams(n,\ m)) \leq 1$
$\land\ \forall\, sid \in HandshakeStreams(n,\ m) :$
$\text{IF}\ sid[1] = n\ \text{THEN}\ nodes[n].conn[m] = \text{“initiator”}$
$\text{ELSE}\ \ nodes[m].conn[n] = \text{“initiator”}$

A peer opens no stream before sending its *init* record, and no announcement stream before its handshake completes (draft: “Connection Handshake”).

$HandshakeBeforeStreams \triangleq$
$\forall\, n \in InitialPeers : \forall\, m \in OtherPeers[n] :$
$\land\ \neg nodes[n].init\_sent[m] \Rightarrow$
$\forall\, k \in 1\ ..\ MaxStreams : S(n,\ m,\ \langle n,\ k\rangle).status = \text{“none”}$
$\land\ \neg HandshakeComplete(n,\ m) \Rightarrow$
$\forall\, k \in 1\ ..\ MaxStreams : S(n,\ m,\ \langle n,\ k\rangle).rtype \notin AnnTypes$

Once n has received m's *init*, the negotiated version is the minimum of the two advertised versions (draft: “Protocol *Versioning*”).

$NegotiatedVersionIsMin \triangleq$
$\forall\, n \in InitialPeers : \forall\, m \in OtherPeers[n] :$
$HandshakeComplete(n,\ m) \Rightarrow$
$nodes[n].version[m] = Min(nodes[n].my\_version,\ nodes[m].my\_version)$

A sender never has two announcement streams of one type open toward a peer (draft: “Announcement Streams”).

$SenderSingleton \triangleq$
$\forall\, n \in InitialPeers : \forall\, m \in OtherPeers[n] : \forall\, t \in AnnTypes :$
$Cardinality(LiveAnn(n,\ m,\ t)) \leq 1$

OBSOLETE is only ever used against a peer that advertised an old version.

$$
\begin{array}{l}
\mathit{CloseJustified} \triangleq \\
\quad \forall\, n \in \mathit{InitialPeers} : \forall\, m \in \mathit{OtherPeers}[n] : \\
\qquad \mathit{nodes}[n].\mathit{close}[m] = \text{“OBSOLETE”} \Rightarrow \\
\qquad\quad \lor\ \mathit{nodes}[n].\mathit{my\_version} < \mathit{MinVersion} \\
\qquad\quad \lor\ \mathit{nodes}[m].\mathit{my\_version} < \mathit{MinVersion}
\end{array}
$$

Blocks are always a contiguous chain from genesis.

$$
\begin{array}{l}
\mathit{BlocksContiguous} \triangleq \\
\quad \forall\, n \in \mathit{InitialPeers} : \mathit{nodes}[n].\mathit{blocks} = 1 \,..\, \mathit{Height}(n)
\end{array}
$$

Request streams are only opened after the handshake completes, one at a time per peer (draft: “Connection Handshake”; $ZIP-204$ in-transit limit).

$$
\begin{array}{l}
\mathit{RequestsAfterHandshake} \triangleq \\
\quad \forall\, n \in \mathit{InitialPeers} : \forall\, m \in \mathit{OtherPeers}[n] : \\
\qquad \land\ \mathit{OpenReq}(n,\, m) \neq \{\} \Rightarrow \mathit{HandshakeComplete}(n,\, m) \\
\qquad \land\ \mathit{Cardinality}(\mathit{OpenReq}(n,\, m)) \leq 1
\end{array}
$$

Responses in flight respect the draft’s bounds: at most *MaxHeaders* contiguous headers, at most *MaxBlocksPerRequest* hashes per get-blocks.

$$
\begin{array}{l}
\mathit{ResponsesBounded} \triangleq \\
\quad \forall\, n \in \mathit{InitialPeers} : \forall\, m \in \mathit{OtherPeers}[n] : \forall\, \mathit{sid} \in \mathit{StreamIds}(n,\, m) : \\
\qquad \forall\, i \in 1 \,..\, \mathit{Len}(S(n,\, m,\, \mathit{sid}).\mathit{inq}) : \\
\qquad\quad \text{LET } x \triangleq S(n,\, m,\, \mathit{sid}).\mathit{inq}[i] \\
\qquad\quad \text{IN } \quad \land\ x.\mathit{kind} = \text{“headers”} \quad \Rightarrow \mathit{Len}(x.\mathit{heights}) \leq \mathit{MaxHeaders} \land \mathit{Contiguous}(x.\mathit{heights}) \\
\qquad\qquad\quad\ \ \land\ x.\mathit{kind} = \text{“get-blocks”} \Rightarrow \mathit{Len}(x.\mathit{heights}) \leq \mathit{MaxBlocksPerRequest} \\
\qquad\qquad\quad\ \ \land\ x.\mathit{kind} = \text{“blocks”} \quad \Rightarrow \mathit{Len}(x.\mathit{heights}) \leq \mathit{MaxBlocksPerRequest}
\end{array}
$$

Honest peers never earn a misbehavior penalty.

$$
\begin{array}{l}
\mathit{NoHonestPenalty} \triangleq \\
\quad \forall\, n \in \mathit{InitialPeers} : \forall\, m \in \mathit{OtherPeers}[n] : \mathit{nodes}[n].\mathit{score}[m] = 0
\end{array}
$$

No interleaving of conformant behaviour ends in a protocol error. There are no adversarial actions in this phase, so any close other than *OBSOLETE* is a disagreement between honest peers about the draft’s rules.

$$
\begin{array}{l}
\mathit{NoHonestProtocolError} \triangleq \\
\quad \forall\, n \in \mathit{InitialPeers} : \forall\, m \in \mathit{OtherPeers}[n] : \\
\qquad \mathit{nodes}[n].\mathit{close}[m] \in \{\mathit{NoCode},\ \text{“OBSOLETE”}\}
\end{array}
$$